

# Unlocking Software-defined GPU Fabric Scheduling in the LLM Era

Paper #33, 6 pages + references

## Abstract

Large language model (LLM) systems increasingly rely on aggressive optimizations such as prefill–decode (PD) disaggregation, KV cache offloading, and compute–communication overlap to maximize hardware utilization and meet strict latency and throughput requirements. However, these techniques implicitly assume that the underlying GPU interconnects behave as neutral communication substrates. In practice, modern GPU clusters contain shared arbitration domains across PCIe, NVLink/NVSwitch, and RDMA fabrics, where concurrent communication and data movement contend for the same resources. In this paper, we characterize fabric contention in modern GPU clusters and find that it can expose implicit—and sometimes skewed—hardware priorities among traffic classes, leading to performance behavior that deviates from application demands. In particular, GPU-to-GPU communication over NVLink/NVSwitch can contend with local GPU memory access and slow compute progress during communication–computation overlap, while RDMA communication can dominate PCIe arbitration and substantially degrade host–device data movement. To address this challenge, we propose GPUWeaver, a lightweight runtime framework for microarchitecture-aware communication scheduling. GPUWeaver monitors application progress and fabric utilization, detects emerging contention, and dynamically regulates communication injection to better align resource usage with application demands.

## 1 Introduction

Large language models (LLMs) are pushing GPU clusters into a new regime where the GPU fabric is a first-class system resource [21, 26]. Unlike traditional DL workloads with moderate, periodic transfers, LLM pipelines generate continuous, high-volume traffic— $O(100)$  GB/s– $O(1)$  TB/s—across heterogeneous links (NVLink/NVSwitch, PCIe, and GPUDirect RDMA) [20, 33]. LLMs also expand the working set from “small and mostly static” to “large and stateful” (Table 1): beyond GB-scale parameters, they maintain hundreds of GBs of runtime state (e.g., KV cache) that must be created and accessed at runtime [12, 23]. This breaks the prior assumption that a single GPU (e.g., H100) can easily hold a whole AI model (e.g., ResNet-50) [9, 19, 26]. As a result, LLM workloads increasingly depend on how efficiently the system places and transfers parameters and state across GPUs and hosts—requiring a fabric that offers not just bandwidth, but also predictability and hardware efficiency.

This trend has led to many innovations in LLM infrastructure, ranging from disaggregation [22, 33], parallelisms [16, 26], GPU compute-communication overlapping [32], to collective communication optimizations [25, 34]. However, these

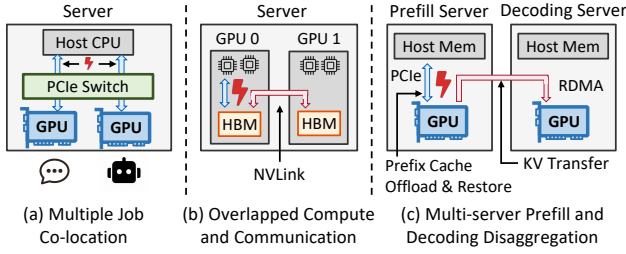
| Model / HW         | Params  | Runtime State  | Takeaway         |
|--------------------|---------|----------------|------------------|
| ResNet-50(DL)      | 45MB    | Act@2048: 4GB  | Small & static   |
| Llama3.1-405B(LLM) | 810GB   | KV@128K: 123GB | Large & stateful |
| Qwen3-Max(LLM)     | >2000GB | KV@262K: 88GB  | Large & stateful |
| H100(GPU)          | L2 50MB | HBM 80GB       | –                |

**Table 1: DL vs. LLM footprint shift (BF16/FP16).**

efforts assume that the underlying GPU fabric can be treated as a best-effort black box. This creates a context gap between what LLM workloads need—service level objectives (SLO) and phase-specific goals in LLM pipelines—and what the fabric actually provides: shared hardware resources with complex contention behaviors (Figure 1). Without principled coordination, LLM runtimes could show high nominal GPU activity yet still experience utilization cliffs, unpredictable slowdown, and costly overprovisioning in AI datacenters.

In this paper, we argue for a vision we call software-defined GPU fabric scheduling: modeling GPU fabric use as an explicit scheduling problem—not an emergent side effect—so LLM jobs and their pipeline phases can coordinate to optimize global objectives. Basically, LLM tasks (or phases) are not uniformly urgent: tasks with slack (e.g., background state movement) can yield GPU fabric resources to latency-critical tasks with tight deadlines, enabling SLO-aware sharing rather than best-effort contention. Realizing this vision requires elevating the GPU fabric from “plumbing” to a managed resource, much like how OS schedulers manage CPU and memory rather than leaving it to uncoordinated processes. Concretely, GPU fabric should expose control knobs (e.g., NVLink pacing) and valuable signals (e.g., GPU memory bandwidth usage) to a scheduler, and the scheduler should be able to automatically adjust control knobs reacting to relevant signals, driven by the objectives of LLM tasks. This would allow LLM tasks to share GPU fabric resources cooperatively, delivering predictable LLM performance without sacrificing high GPU utilization.

Achieving this vision raises several challenges. First, we must explain why current LLM deployments struggle: identify when LLM jobs or phases violate their goals and attribute slowdowns to specific root causes in GPU fabric resource contention across LLM tasks and phases. This requires a deep understanding on the intrinsic microarchitectural behaviors of GPU fabrics under realistic LLM I/O patterns. Second, we must connect root causes to reusable primitives: determine which signals are predictive and practical to gather, and which control knobs are feasible to enforce. Unlike CPU scheduling, not all GPU fabric resources expose direct actionable controls; an effective design must combine direct controls where available with indirect knobs such as pacing and traffic shaping at appropriate software boundaries. Finally, we need a mapping layer that translates global and



**Figure 1: Resource contentions in LLM deployments.**

LLM task specific objectives (e.g., SLOs and phase properties) into timely microarchitecture-level scheduling strategies that can be enforced for existing LLM stacks transparently.

We present GPUWeaver, a lightweight runtime framework toward this vision, grounded in two kinds of GPU fabric contentions that increasingly dominate LLM deployments (§3): high-bandwidth memory (HBM) contention, where NVLink-driven accesses interfere with local GPU memory traffic and silently throttle compute, and PCIe contention, where GPUDirect RDMA competes with GPU↔CPU data movement (e.g., KV cache CPU offload), breaking overlap assumptions and amplifying tail latency with degraded throughput. Based on these insights, GPUWeaver (1) offers an architectural overview of the programmable GPU fabric scheduling solution through a software-hardware co-design principle (§4.1); (2) proposes a set of novel and reusable signal and action primitives to mitigate GPU fabric contentions (§4.2); and (3) demonstrates a case study where LLM task-level objectives could be translated into hardware scheduling strategies (§4.3).

## 2 Background

In this section, we further demonstrate the trend of GPU fabric resource sharing in LLM stacks, discuss the major limitations of existing efforts, and explore an operating point—software-defined GPU fabric scheduling.

### 2.1 GPU Fabric Sharing in LLM Workloads

To improve hardware utilization, today’s LLM stacks increasingly attempt many system-level innovations. These efforts include offloading the runtime state (e.g., KV cache) from GPU high bandwidth memory (HBM) to host memory [4, 11, 27, 29], colocating multiple LLM instances on the same multi-GPU server (e.g., a chatbot alongside a document assistant in Figure 1(a)) [6, 8, 30], overlapping compute and communication for LLM operator parallelisms (Figure 1(b)) [15, 31, 32], and disaggregating inference into separate prefill and decode workers that may run on different GPUs or even different servers (Figure 1(c)) [22, 33]. These efforts inherently lead to *GPU fabric sharing* across tasks with various objectives: prefill/decode tasks and chatbot requests are typically *latency-critical* (the lower the latency, the better), while background activities, such as KV cache offload,

and model refresh with checkpointing, are often best-effort yet still consume substantial GPU fabric resources.

When co-existing tasks share the same GPU fabric, they inevitably interfere with one another as they concurrently contend for shared hardware resources, such as GPU HBM and PCIe links. In particular, GPU HBM can be frequently accessed by local GPU compute units for LLM operator compute, or by NVLink for moving state to a neighbor GPU; One PCIe link is used both for GPU↔CPU transfers and for GPU↔RDMA NIC (RNIC) transfers used by GPUDirect RDMA. When LLM tasks make concurrent use of them, these components can independently cause resource contentions. For example, a latency-sensitive RDMA-based KV transfer between prefill and decode servers may be colocated with a periodic best-effort KV cache offload from GPU to CPU memory. Because the offload task can consume a substantial fraction of PCIe bandwidth, it can significantly hinder the KV transfer from meeting its performance objective.

### 2.2 Why Existing Approaches Fall Short

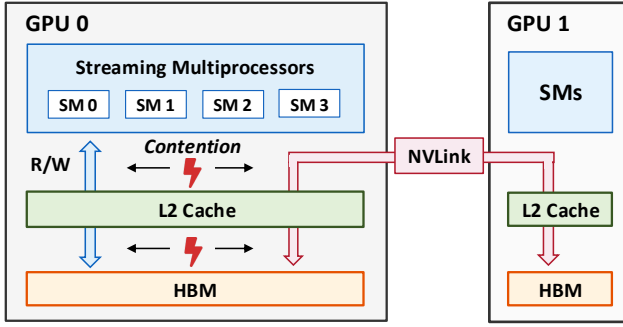
Despite rapid progress in LLM systems, most systems remain *workload-centric*. They optimize overlap and execution scheduling within a job [3, 10, 13], yet largely treat PCIe/NVLink fabrics as a black box [2], and thus lack a principled design to orchestrate various LLM tasks. This exposes three limitations:

**(1) Limited fabric-centric control.** Existing schedulers primarily reason about *which* compute/communication stages to run and *when* [17], but rarely control *how* different traffic classes enter shared arbitration domains (e.g., PCIe switch hierarchies, NVLink/NVSwitch paths, or the GPU memory system). As a result, they cannot explicitly coordinate competing flows on the shared fabric and must rely on indirect workload-level decisions [2].

**(2) Opaque and largely immutable arbitration.** Once requests are injected into a shared domain (e.g., the PCIe hierarchy or GPU memory subsystem), effective service order and priority are largely determined by hardware mechanisms. Practical Quality-of-Service (QoS) controls are limited and often insufficient to enforce application-level criticality under contention [24, 28].

**(3) Reactive, coarse-grained workarounds.** Prior approaches mainly mitigate interference by blocking overlap, throttling at coarse granularity, or applying stream-level prioritization, rather than *scheduling the fabric* itself to recover latent bandwidth and align resource allocation with demand [2, 5, 13, 14].

**A new operating point: GPU fabric scheduling.** These limitations motivate a *software-defined GPU fabric scheduling framework* called GPUWeaver that (i) allows LLM tasks to express explicit intent (e.g., performance objectives and QoS level), (ii) timely perceives impending resource contentions and reasons about their root causes, and (iii) applies



**Figure 2: Contention between memory access and NVLink transfer.**

scheduling policies through software-visible control knobs to improve fabric sharing efficiency.

### 3 Fabric Contentions

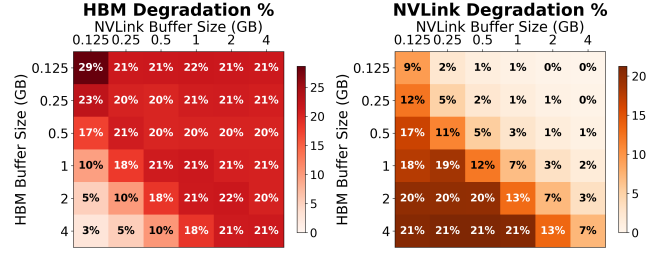
In this section, we present two forms of fabric contention that we observe in modern GPU clusters, along with detailed measurements and case studies demonstrating their potential impact on real-world AI workloads.

#### 3.1 GPU Memory Contention

Computation in the streaming multiprocessor (SM) involves memory requests for GPU’s HBM. Inter-GPU communication is enabled by NVLink, where dedicated DMA copy engines are responsible for transferring data between the HBM and the NVLink interface in both directions. Consequently, computation and communication inject traffic into the same on-chip interconnect and off-chip HBM, leading to resource contention [7], as illustrated in Figure 2.

**Root cause:** Contention emerges because memory requests issued by SMs and DMA transfers from copy engines compete for shared resources within a unified arbitration domain. First, the interconnect fabric of L2 cache provides limited bandwidth and constrained arbitration capacity, such that simultaneous bursts from SMs and DMA engines are competing at L2 fabric. Second, memory controllers maintain request queues with fixed capacity. Co-locating SM and DMA transactions within these queues can generate back-pressure and pipeline stalls as occupancy increases, thereby degrading effective throughput for both traffic streams.

**Contention characterization.** We characterize GPU memory contention by concurrently executing a memory access kernel and an NVLink communication kernel on a DGX-H100 server. Both kernels are launched in parallel on GPU A using separate CUDA streams to enable overlap. The memory access kernel generates interleaved read and write operations to contiguous memory locations, specifically loading a 128-byte cache line from HBM and subsequently writing it back to the same region, thereby sustaining streaming memory traffic. The NVLink communication kernel invokes



**Figure 3: HBM and NVLink effective bandwidth degradation under contention.**

the CUDA runtime API `cudaMemcpyPeerAsync` to perform a bidirectional peer-to-peer data transfer between GPU A and GPU B.

We sweep both memory access and NVLink transfer buffer sizes from 128 MB to 4 GB. The bi-directional NVLink transfer uses such buffer size in each direction. Specifically, we evaluate a two-dimensional grid in which the memory access size and the NVLink transfer size independently take values from 128 MB to 4 GB. For each (HBM, NVLink) pair, three scenarios are measured: (1) an isolated HBM baseline with only the memory access kernel active, (2) an isolated NVLink baseline with only the NVLink communication kernel active, and (3) a concurrent execution in which both kernels run simultaneously. Contention is reported as a percentage degradation,

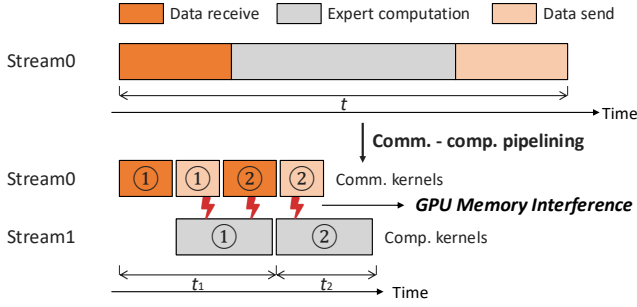
$$\text{Degradation}(\%) = 100 \times \left( 1 - \frac{\text{BW}_{\text{contended}}}{\text{BW}_{\text{isolated}}} \right),$$

where normalization is performed against the corresponding isolated baseline for the metric under consideration.

The results are shown in Figure 3. Across the  $6 \times 6$  sweep (128 MB–4 GB), the two heatmaps indicate that contention is primarily governed by the temporal overlap between the GPU’s local memory access and NVLink transfer.

**HBM degradation** exhibits a pronounced triangular plateau. When the NVLink transfer is comparable to or longer than the memory access workload, corresponding to the upper-right region and near the main diagonal, HBM throughput decreases to a stable loss of approximately 20%, reflecting a steady-state regime under sustained copy-engine pressure. In contrast, when the NVLink transfer is short relative to the memory access kernel, corresponding to the lower-left region, HBM degradation is limited to roughly 3–10%, as the overlap interval is short.

**NVLink degradation** shows a complementary, approximately transposed pattern. It reaches about 20% when a short transfer is entirely overlapped by a long-running memory access kernel, but declines to approximately 0–1% when the memory access kernel completes early and the transfer proceeds largely without contention. Along the matched-duration diagonal for sizes  $\geq 256$  MB, HBM consistently experiences greater degradation than NVLink, with NVLink



**Figure 4: GPU memory contention in compute-communication pipelining optimization.**

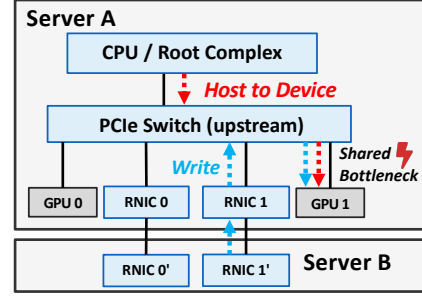
typically limited to around 5–7% loss. This asymmetry suggests that NVLink traffic retains an effective service advantage under full overlap. The elevated 29% degradation observed at 128 MB likely reflects small working-set effects, such as L2-sensitive or non-steady-state behavior, which fall outside the stable streaming regime.

**Case study.** During inference with Mixture-of-Experts (MoE) models, each token must be *dispatched* to selected experts within the feed-forward network (FFN) layer and subsequently *combined* after FFN execution. The dispatch and combine stages require *all-to-all* communication, generating bidirectional NVLink traffic in which each GPU simultaneously sends and receives tokens. Meanwhile, expert FFN computation imposes sustained HBM read and write pressure due to frequent accesses to weights and activations [15, 31, 32]. As both communication and computation streams demand substantial HBM bandwidth, overlapping these operations can induce the contention effects characterized above, as illustrated in Figure 4.

### 3.2 PCIe Contention

Modern GPU servers place GPUs and RDMA NICs under the same PCIe hierarchy. As Figure 5 shows, two traffic classes may concurrently share the PCIe: (i) *GPU↔CPU* host-device data movement (e.g., H2D/D2H copies, paging, and control traffic), and (ii) *NIC↔GPU* communication via GPUDirect RDMA (GDR), where the NIC directly reads/writes GPU memory. PCIe provides much lower bandwidth than HBM, and thus is more likely to become a bottleneck when host-device movement and RDMA traffic occur simultaneously. For example, NVIDIA A100 delivers up to TB/s-level HBM bandwidth, whereas a PCIe Gen4 ×16 interface is only on the order of tens of GB/s [18]. When multiple activities contend on the same PCIe, transfers with more urgent bandwidth demand are more likely to be impacted and become bottlenecked.

**Root cause: contention on a shared PCIe segment.** As Figure 5 illustrates, the *RDMA write* stream is initiated by Server B and writes directly into Server A’s GPU memory via GDR, while the *H2D* stream is generated on Server A by its



**Figure 5: PCIe contention topology.** Blue: NIC→GPU GPUDirect RDMA write. Red: CPU→GPU H2D. Both contend on the same GPU-side PCIe segment.

CPU staging data to the GPU. Both streams traverse the same GPU-side PCIe switch/uplink and thus contend on a shared segment. Similarly, RDMA read and D2H follow analogous paths in the reverse directions and also traverse the reverse GPU-side PCIe switch/uplink, leading to contention.

**Contention characterization.** We characterize PCIe contention using a two-server RDMA setup. On Server A, we generate sustained host-device copy traffic using pinned host memory and `cudaMemcpyAsync` to saturate either D2H or H2D. Concurrently, a remote Server B issues GDR read/write requests to Server A’s GPU memory.

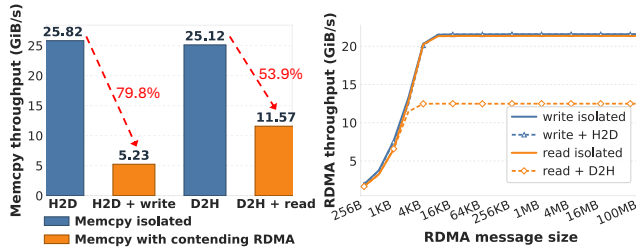
We first measure steady-state contention under large RDMA messages. Figure 6 (left) reveals a striking asymmetry: under concurrent GDR write traffic, H2D bandwidth drops from 25.82 GiB/s to 5.23 GiB/s (79.8% degradation); while under GDR read traffic, D2H bandwidth decreases from 25.12 GiB/s to 11.57 GiB/s (53.9% degradation).

Next, we examine the reverse direction by quantifying how sustained host–device transfers affect GDR performance across message sizes. We run four cases: (1) GDR write isolated, (2) GDR write with background H2D, (3) GDR read isolated, and (4) GDR read with background D2H. Figure 6 (right) shows a strong direction-dependent asymmetry: background H2D has negligible impact on GDR write throughput, while background D2H sharply reduces GDR read throughput and stabilizes at ~41.5% degradation when message sizes are ≥8 KB.

Overall, these results reveal an *implicit priority skew on PCIe that varies across traffic types and directions*. This skew originates from hardware design and cannot be changed once the device is fabricated, and may prevent demanding tasks from obtaining sufficient bandwidth. Since the PCIe segments often lie on the critical path of workloads, such skew can amplify end-to-end slowdown though the link is highly utilized. This motivates software mechanisms to flexibly protect demanding tasks under PCIe concurrent traffic.

**Case studies.** *Offloading in PD-disaggregated pipelines.* As discussed in §2, modern LLM systems increasingly rely on offloading state from GPU HBM to host CPU DRAM and restoring it on demand. PD-disaggregated serving places prefill and





**Figure 6: PCIe contention characterization. Left: large-RDMA impact on host-device copies. Right: RDMA throughput under four cases.**

decode on different engines or nodes, introducing inter-node communication via RDMA to move runtime state across the fabric [1, 22, 33]. As a result, RDMA state movement and host-device copies contend on shared PCIe resources, which can slow latency-critical traffic (e.g., KV staging/restoration) and inflate serving latency such as TTFT [23, 28, 29].

*Multi-tenant serving with priority heterogeneity.* As discussed in §2, modern serving stacks increasingly multiplex heterogeneous LLM workloads on the same GPU servers to improve utilization [6, 8, 30]. Latency-critical interactive workloads and best-effort background jobs may overlap in execution and share the same PCIe fabric (e.g., during KV cache offloading from CPU memory through the PCIe switch to GPUs). When such activities overlap, their data movements contend within the same PCIe arbitration domain.

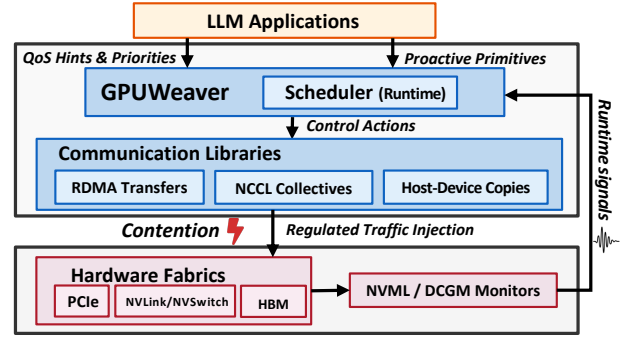
Together, these trends highlight a growing need for *fabric-aware mixed-criticality control*. In modern LLM deployments, different traffic classes sharing PCIe often have very different performance sensitivities. This motivates exposing a lightweight *software-defined knob* to regulate traffic injection across PCIe, enabling better serving performance under concurrent traffic.

## 4 GPUWeaver framework

To mitigate fabric-level contention without requiring hardware modifications, we propose GPUWeaver, a lightweight runtime framework for microarchitecture-aware communication scheduling in GPU clusters. The key idea is to regulate the effective injection behavior of communication flows, controlling how aggressively different traffic classes enter shared fabrics such as PCIe and NVLink/NVSwitch.

### 4.1 System Overview

Figure 7 illustrates the architecture of GPUWeaver. The runtime sits between user-space LLM applications and the underlying communication libraries, acting as a control layer that observes runtime behavior and regulates communication injection. GPUWeaver combines two complementary mechanisms. First, it provides an automatic scheduler that regulates



**Figure 7: System overview of GPUWeaver.**

communication based on runtime signals and application-provided QoS hints. These hints specify coarse-grained priorities among traffic classes (e.g., latency-critical host-device transfers versus best-effort communication), allowing the scheduler to make global decisions about how aggressively different flows should inject traffic into shared fabrics. Second, GPUWeaver exposes a small set of primitives that allow applications or runtimes to proactively adjust communication priorities when necessary. This interface is particularly useful for short-lived or bursty phases where purely reactive control may be too slow to respond.

GPUWeaver takes two sources of inputs. The first consists of application-level priorities and QoS hints that describe the relative importance of different communication activities. The second consists of runtime signals collected during execution, including indicators of fabric pressure and application progress. Together, these inputs allow the system to infer when contention arises across shared resources such as PCIe or NVLink/NVSwitch.

Based on these inputs, a centralized scheduler periodically evaluates the current system state and determines whether communication injection should be adjusted. The scheduler then applies a set of control actions that regulate how communication operations are issued to the underlying communication stacks, including RDMA transfers, NCCL collectives, host-device memory copies, etc. During normal execution, communication proceeds without intervention. When runtime signals indicate emerging contention, or when applications proactively trigger priority adjustments, the scheduler modifies the injection behavior of competing flows by tuning communication parameters such as pacing, batching, or overlap with computation. In this way, GPUWeaver reshapes communication pressure on shared fabrics without requiring modifications to hardware arbitration mechanisms.

### 4.2 Contention Signals and Control Knobs

GPUWeaver collects lightweight runtime signals capturing both fabric pressure and application progress. These signals are obtained through interfaces such as NVML and DCGM, allowing the system to operate with negligible overhead.

More specifically, for intra-node contention, GPUWeaver monitors NVLink/NVSwitch TX/RX bandwidth alongside GPU HBM bandwidth usage, which together indicate when GPU computation and GPU-to-GPU communication compete for shared memory resources. When NVLink traffic is high while GPU compute progress slows, the system infers contention within the GPU memory system. For PCIe contention, GPUWeaver monitors GPU-side PCIe throughput together with NIC traffic statistics to estimate how RDMA transfers and host-device memory copies share the PCIe hierarchy. When PCIe activity approaches saturation and host-device transfers slow down, the system infers that RDMA injection dominates the link.

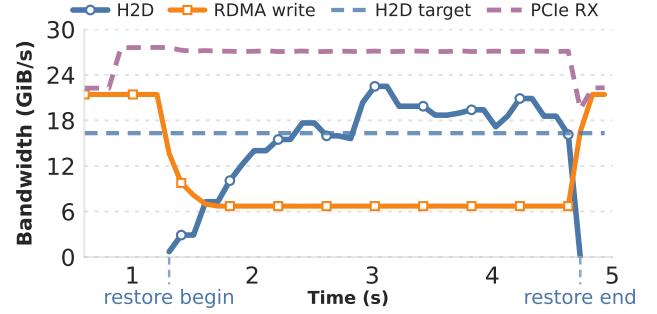
Based on these signals, or when proactively triggered by applications, GPUWeaver adjusts a set of software-level control knobs that regulate communication injection. For GPU-to-GPU communication, the system can regulate the overlap between communication and computation or adjust the batching behavior of collective operations. For RDMA traffic, it can pace or batch RDMA operations to limit NIC-driven injection into PCIe. These mechanisms allow GPUWeaver to smooth communication bursts and prevent individual traffic classes from dominating shared fabrics.

Despite targeting different fabrics, these mechanisms follow a unified abstraction: regulating how aggressively communication operations are injected into shared resources. By shaping communication injection rather than relying solely on hardware arbitration, GPUWeaver allows applications and runtimes to influence how competing traffic classes share the fabric.

### 4.3 Case Study: KV Cache Offloading

To illustrate the feasibility of GPUWeaver, we implement a preliminary prototype for PD-disaggregated KV offloading as a concrete case study. In this setting, a decode worker restores KV state from host memory to GPU HBM while concurrent RDMA traffic simultaneously moves state across nodes. As discussed in § 3.2, these two activities share the same GPU-side PCIe segment, so a KV restore can be delayed even when overall PCIe utilization remains high. Also, §3.2 showed that under concurrent GDR write traffic, H2D bandwidth can collapse from 25.82 GiB/s to 5.23 GiB/s. In this case, GPUWeaver treats the restore interval as a protected window and maps this application-level priority into explicit PCIe-side control.

When a restore window begins, the runtime marks the corresponding H2D copies as critical, and the scheduler monitors GPU-side PCIe RX activity together with H2D progress. Rather than accepting the hardware-imposed operating point, the scheduler maintains an explicit H2D target and treats any sustained drop below that target under high PCIe demand as harmful contention. In Figure 8, we set the target to 16.3 GiB/s. This target is a policy choice and can be adjusted based on application needs. Once H2D falls below



**Figure 8: PCIe case study. The scheduler steers H2D toward a configurable target by suppressing competing RDMA writes during the restore window.**

the target, GPUWeaver imposes the injection-shaping action described in § 4.2, progressively throttling the competing RDMA writes using software-visible pacing and queue-depth controls. When the protected window ends, GPUWeaver relaxes these limits and returns to a work-conserving mode.

Figure 8 shows a representative run. The scheduler suppresses the competing RDMA writer from roughly 21 GiB/s to about 7 GiB/s within the first 0.6 s of the protected window, after which H2D stays near the configured target. Relative to the 5.23 GiB/s H2D throughput observed under contended GDR write in § 3.2, this recovers roughly 11–13 GiB/s of bandwidth during the restore window. This preliminary prototype shows how a serving objective can be translated into concrete, fabric-aware control, while motivating future work on richer signals, faster response times, and evaluation in a full serving system.

## 5 Conclusion & Future Work

This paper studies fabric-level contention in modern GPU clusters and shows that communication and computation can interact in subtle yet impactful ways across shared fabrics. Through detailed characterization, we identify two common contentions: contention between NVLink communication and GPU HBM access, and contention between GPUDirect RDMA and host-device transfers on PCIe. Our results reveal that these fabrics exhibit implicit priority behaviors, where certain traffic classes can disproportionately throttle others despite high overall bandwidth availability. We further discuss how such effects can arise in emerging LLM workloads.

Looking forward, we plan to extend our study to quantify the end-to-end impact of these fabric contentions on real LLM workloads through more comprehensive case studies. We also aim to further explore the design space of runtime signals and control knobs for regulating communication injection, identifying practical mechanisms that can effectively mitigate contention with minimal overhead. Ultimately, we hope this work motivates treating GPU fabric behavior as a first-class system concern and encourages the development of software-defined approaches for managing compute-communication interaction in large-scale AI systems.

## References

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134, 2024.
- [2] Jiamin Cao, Yu Guan, Kun Qian, Jiaqi Gao, Wencong Xiao, Jianbo Dong, Binzhang Fu, Dennis Cai, and Ennan Zhai. Crux: GPU-Efficient Communication Scheduling for Deep Learning Training. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 1–15, Sydney NSW Australia, August 2024. ACM.
- [3] Chang Chen, Xiuhong Li, Qianchao Zhu, Jiangfei Duan, Peng Sun, Xingcheng Zhang, and Chao Yang. Centauri: Enabling Efficient Scheduling for Communication-Computation Overlap in Large Model Training via Communication Partitioning. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 178–191, La Jolla CA USA, April 2024. ACM.
- [4] Yihua Cheng, Yuhua Liu, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Kuntai Du, and Junchen Jiang. Lmcache: An efficient kv cache layer for enterprise-scale llm inference. *arXiv preprint arXiv:2510.09665*, 2025.
- [5] Daniele De Sensi, Lorenzo Pichetti, Flavio Vella, Tiziano De Matteis, Zebin Ren, Luigi Fusco, Matteo Turisini, Daniele Cesarini, Kurt Lust, Animesh Trivedi, Duncan Roweth, Filippo Spiga, Salvatore Di Girolamo, and Torsten Hoefer. Exploring GPU-to-GPU Communication: Insights into Supercomputer Interconnects. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15, Atlanta, GA, USA, November 2024. IEEE.
- [6] Jiangfei Duan, Runyu Lu, Haojie Duanmu, Xiuhong Li, Xingcheng Zhang, Dahua Lin, Ion Stoica, and Hao Zhang. Muxserve: flexible spatial-temporal multiplexing for multiple llm serving. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- [7] Paul Elvinger, Foteini Strati, Natalie Enright Jerger, and Ana Klimovic. Understanding gpu resource interference one level deeper. In *Proceedings of the 2025 ACM Symposium on Cloud Computing, SoCC ’25*, page 687–694, New York, NY, USA, 2026. Association for Computing Machinery.
- [8] Shiwei Gao, Qing Wang, Shaoxun Zeng, Youyou Lu, and Jiwei Shu. Weaver: Efficient {Multi-LLM} serving with attention offloading. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*, pages 587–595, 2025.
- [9] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [10] Ke Hong, Xiuhong Li, Minxu Liu, Qiuli Mao, Tianqi Wu, Zixiao Huang, Lufang Chen, Zhong Wang, Yichong Zhang, Zhenhua Zhu, Guohao Dai, and Yu Wang. Efficient and adaptable overlapping for computation and communication via signaling and reordering. In *Proceedings of the European Conference on Computer Systems (EuroSys)*, 2026.
- [11] Xuanlin Jiang, Yang Zhou, Shiyi Cao, Ion Stoica, and Minlan Yu. Neo: Saving gpu memory crisis with cpu offloading for online llm inference. *Proceedings of Machine Learning and Systems*, 7, 2025.
- [12] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, page 611–626, 2023.
- [13] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. Accelerating distributed MoE training and inference with lina. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 945–959, Boston, MA, 2023. USENIX Association.
- [14] Bowen Liu, Xinyang Huang, Qijing Li, Zhuobin Huang, Yijun Sun, Wenxue Li, Junxue Zhang, Ping Yin, and Kai Chen. Ceio: A cache-efficient network i/o architecture for nic-cpu data paths. In *Proceedings of the ACM SIGCOMM 2025 Conference*, pages 381–394, 2025.
- [15] Ziming Mao, Yihan Zhang, Chihan Cui, Zhen Huang, Kaichao You, Zhongjie Chen, Zhiying Xu, Zhenyu Gu, Scott Shenker, Costin Raiciu, Yang Zhou, and Ion Stoica. Uccle: Portable expert-parallel communication. *arXiv preprint arXiv:2512.19849*, 2026.
- [16] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. Pipedream: generalized pipeline parallelism for dnn training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles, SOSP ’19*, page 1–15, New York, NY, USA, 2019. Association for Computing Machinery.
- [17] Kelvin K. W. Ng, Henri Maxime Demoulin, and Vincent Liu. Paella: Low-latency model serving with software-defined gpu scheduling. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP ’23*, page 595–610, New York, NY, USA, 2023. Association for Computing Machinery.
- [18] NVIDIA. NVIDIA A100 Tensor Core GPU Architecture (Ampere Architecture Whitepaper). <https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf>, 2020.
- [19] NVIDIA. NVIDIA H100 Tensor Core GPU Datasheet. <https://resources.nvidia.com/en-us-hopper-architecture/nvidia-tensor-core-gpu-datasheet>, 2024.
- [20] NVIDIA. DGX SuperPOD Architecture. <https://docs.nvidia.com/dgx-superpod/reference-architecture-scalable-infrastructure-h100/latest/dgx-superpod-architecture.html>, 2025.
- [21] NVIDIA. IMEX Service for NVLink Networks: Overview. <https://docs.nvidia.com/multi-node-nvlink-systems/imex-guide/overview.html>, 2025.
- [22] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture, ISCA ’24*, page 118–132. IEEE Press, 2025.
- [23] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *Proceedings of the 23rd USENIX Conference on File and Storage Technologies*, pages 155–170, 2025.
- [24] Andre Richter, Christian Herber, Thomas Wild, and Andreas Herkersdorf. Resolving performance interference in sr-iov setups with pcie quality-of-service extensions. In *2016 Euromicro Conference on Digital System Design (DSD)*, pages 454–462, 2016.
- [25] Aashaka Shah, Abhinav Jangda, Binyang Li, Caio Rocha, Changho Hwang, Jithin Jose, Madan Musuvathi, Olli Saarikivi, Peng Cheng, Qinghua Zhou, Roshan Dathathri, Saeed Maleki, and Ziyue Yang. Mscl++: Rethinking gpu communication abstractions for cutting-edge ai applications. *arXiv preprint arXiv:2504.09014*, 2025.
- [26] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [27] Xinyi Wan, Penghui Qi, Guangxing Huang, Min Lin, and Jialin Li. PipeOffload: Improving scalability of pipeline parallelism with memory optimization. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning*

- Research*, pages 61942–61955. PMLR, 13–19 Jul 2025.
- [28] Yongtong Wu, Shaoyuan Chen, Yinmin Zhong, Rilin Huang, Yixuan Tan, Wentao Zhang, Liyue Zhang, Shangyan Zhou, Yuxuan Liu, Shunfeng Zhou, et al. Dualpath: Breaking the storage bandwidth bottleneck in agentic llm inference. *arXiv preprint arXiv:2602.21548*, 2026.
  - [29] Zhiqiang Xie, Ziyi Xu, Mark Zhao, Yuwei An, Vikram Sharma Maitlody, Scott Mahlke, Michael Garland, and Christos Kozyrakis. Strata: Hierarchical context caching for long context language model serving. *arXiv preprint arXiv:2508.18572*, 2025.
  - [30] Shan Yu, Jiarong Xing, Yifan Qiao, Mingyuan Ma, Yangmin Li, Yang Wang, Shuo Yang, Zhiqiang Xie, Shiyi Cao, Ke Bao, Ion Stoica, Harry Xu, and Ying Sheng. Prism: Unleashing gpu sharing for cost-efficient multi-llm serving. *arXiv preprint arXiv:2505.04021*, 2025.
  - [31] Shulai Zhang, Ningxin Zheng, Haibin Lin, Ziheng Jiang, Wenlei Bao, Chengquan Jiang, Qi Hou, Weihao Cui, Size Zheng, Li-Wen Chang, et al. Comet: Fine-grained computation-communication overlapping for mixture-of-experts. *Proceedings of Machine Learning and Systems*, 7, 2025.
  - [32] Chenggang Zhao, Shangyan Zhou, Liyue Zhang, Chengqi Deng, Zhean Xu, Yuxuan Liu, Kuai Yu, Jiashi Li, and Liang Zhao. DeepEP: an efficient expert-parallel communication library. <https://github.com/deepseek-ai/DeepEP>, 2025.
  - [33] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, 2024.
  - [34] Yang Zhou, Zhongjie Chen, Ziming Mao, ChonLam Lao, Shuo Yang, Pravein Govindan Kannan, Jiaqi Gao, Yilong Zhao, Yongji Wu, Kaichao You, Fengyuan Ren, Zhiying Xu, Costin Raiciu, and Ion Stoica. An extensible software transport layer for gpu networking. *arXiv preprint arXiv:2504.17307*, 2025.